



# Chapter 13: Overloading

# Objectives

- In this chapter, you will
  - Learn about overloading
  - Become familiar with the restrictions on operator overloading
  - Examine the pointer `this`
  - Learn about `friend` functions
  - Learn how to overload operators as members and nonmembers of a class

# Objectives (cont'd.)

- In this chapter, you will (cont'd.)
  - Discover how to overload various operators
  - Become familiar with the requirements for classes with pointer member variables

# Operators in C++

- The C++ operators (+,-,/,\*,...) work in predefined ways when used with built-in data types.
  - When the **+** operator is used with two **int** variables it performs an integer addition operation (10+20)
  - When the **+** operator is used with two **double** variables it performs a floating-point addition operation (10.5+20.6)

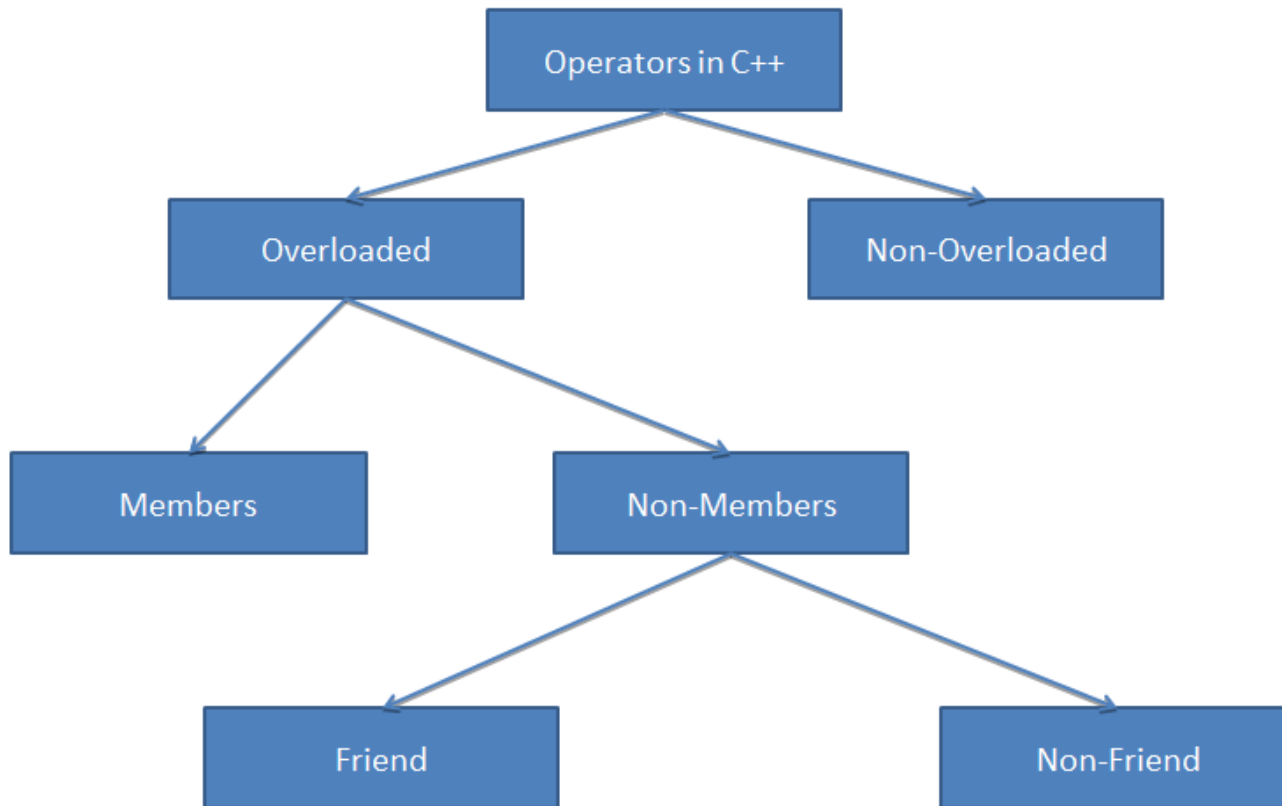
# Operator Overloading

- C++ also allows you to use standard operators (+, -, /, \*, ...) with objects of classes provided that you have to define the exact action that a given operator is to perform.
- Defining the way that an operator works with objects of a particular class is known as **operator overloading**.
  - This defining includes function's header and body

# Why Operator Overloading Is Needed

- Assignment and member selection are the only built-in operations on classes
  - Other operators cannot be applied directly to class objects
- Operator overloading: extends definition of an operator to work with a user-defined data type
  - C++ allows you to extend the definitions of most of the operators to work with classes

# How to Implement Operator Overloading



# Syntax for Operator Functions

- Syntax of an operator function heading:

```
returnType operator operatorSymbol(formal parameter list)
```

- It is a value-returning function
  - `operator` is a reserved word
- To overload an operator for a class:
  - Include operator function declaration in the class definition
  - Write the definition of the operator function



# Operator Overloading as Member Functions

# Overloading as Member Functions: Arithmetic Binary Operators

```
#include<iostream>
using namespace std;
class currency
{
public:
    currency() { dollars=cents=0.0;}
    currency(long dol, long cent)
    {
        setAmount(dol, cent);
    }
    void setAmount(long dol, long cent)
    {
        dollars = dol+cent/100; //adjust for cent amount>=100
        cents    = cent%100;
    }
    void getAmount (long& dol, long& cent)
    {
        dol =dollars;
        cent=cents;
    }
    void print()
    { cout<<'<span>$</span>'<<dollars<<.<span>'</span>'<<cents<<endl;}

private:
    long dollars;
    long cents;
};
```

# Overloading as Member Functions: Arithmetic Binary Operators [cont'd]

- Overloading addition operation (+)

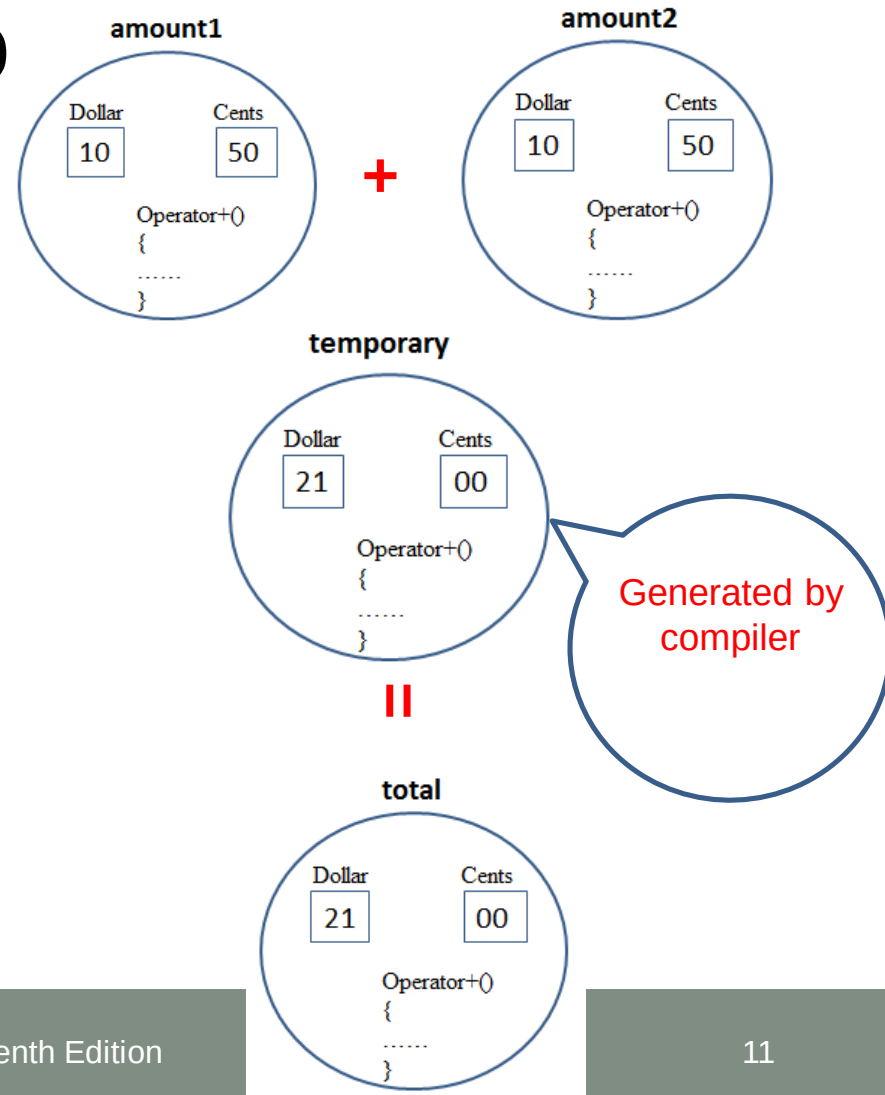
```
//other declaration
currency operator+ (currency r)
{
    currency temp(dollars+r.dollars, cents+r.cents);
    return temp;
}

//other declaration
int main()
{
    currency amount1(10,50), amount2(10,50), total;
    total = amount1 + amount2;
    total.print();

    total = amount1.operator+(amount2);
    total.print();
}
```

## OUTPUT

```
$21.0
$21.0
```



# Overloading as Member Functions: Arithmetic Binary Operators [cont'd]

- **Overloading addition operation (+)**

```
int main()
{
    currency amount1(10,50), amount2(10,50), total;
    total = amount1 + amount2;
    total.print();

    total = amount1.operator+(amount2);
    total.print();

    currency amount3(10,50);
    total = amount1 + amount2 + amount3;
    total.print();
}
```

## OUTPUT

```
$21.0
$21.0
$31.50
```

# Overloading as Member Functions: Arithmetic Binary Operators [cont'd]

- **Overloading addition operation (+) with**

```
//other declaration  
  
currency operator+(long dol)  
{  
    currency temp (dollars+dol,cents);  
    return temp;  
}  
  
//other declaration
```

**OUTPUT**

```
$30.50  
$30.50
```

```
int main()  
{  
    currency amount1(10,50), total;  
    total = amount1 + 20;  
    total.print();  
  
    total= amount1.operator +(20);  
    total.print();  
  
    //total = 20 + amount1;// ERROR: because this st. means "20.operator+ (amount1)".  
    //This error is fixed later using friend function.  
}
```

# Overloading as Member Functions: Binary Operators

- **Overloading [ ]**

```
//other declaration  
  
long operator[](int index)  
{  
    if(index==0)  
        return dollars;  
    else  
        if (index==1)  
            return cents;  
        else  
        {  
            cout<<"ERROR: index out of range"<<endl;  
            exit(1);  
        }  
}  
  
//other declaration
```

## OUTPUT

```
10  
50  
ERROR: index out of range
```

```
int main()  
{  
    currency amount1(10,50);  
    cout<<amount1[0]<<endl;  
    cout<<amount1[1]<<endl;  
    amount1[2];  
}
```

# Overloading as Member Functions: Unary Operators

- **Overloading minus operator (-)**

```
//other declaration

currency operator-()
{
    currency temp (-dollars, cents);
    return temp;
}

//other declaration
```

## OUTPUT

```
$-10.50
$10.50
```

```
int main()
{
    currency amount1(10,50), debt;

    debt=-amount1;
    debt.print();
    amount1.print();
}
```

# Overloading as Member Functions: Unary Operators [cont'd]

- Overloading pre-increment/post-increment (++)

```
//other declaration

currency operator++() // pre-increment
{
    dollars++;
    return *this;
}

currency operator++(int) // post-increment
{
    dollars++;
    return *this;
}

//other declaration
```

```
int main()
{
    currency amount(10,50);
    amount++;
    amount.print();
    ++amount;
    amount.print();
}
```

**OUTPUT**

\$11.50

\$12.50



# Overloading as Member Functions: Unary Operators [cont'd]

- **Overloading (+=) operator**

```
//other declaration

currency operator+=(currency right)
{
    dollars+= right.dollars+ (right.cents+cents)/100;
    cents  = (right.cents+cents)%100;
    return *this;
}

//other declaration

int main()
{
    currency amount1(10,50), amount2(10,50);
    amount1+=amount2;
    amount1.print();
}
```

**OUTPUT**  
\$21.0

# Overloading as Member Functions: Logical Binary Operators

- **Overloading (>) operator**

```
//other declaration  
  
int operator>(currency right)  
{  
    return (dollars*100+cents) > (right.dollars*100+right.cents);  
}  
  
//other declaration
```

```
int main()  
{  
    currency amount1(10,50), amount2(20,50);  
  
    if (amount1>amount2)  
        cout<<"amount1>amount2"<<endl;  
    else  
        cout<<"amount2>amount1"<<endl;  
}
```

**OUTPUT**  
amount2>amount1

# END PART I